

CHAPTER 11.

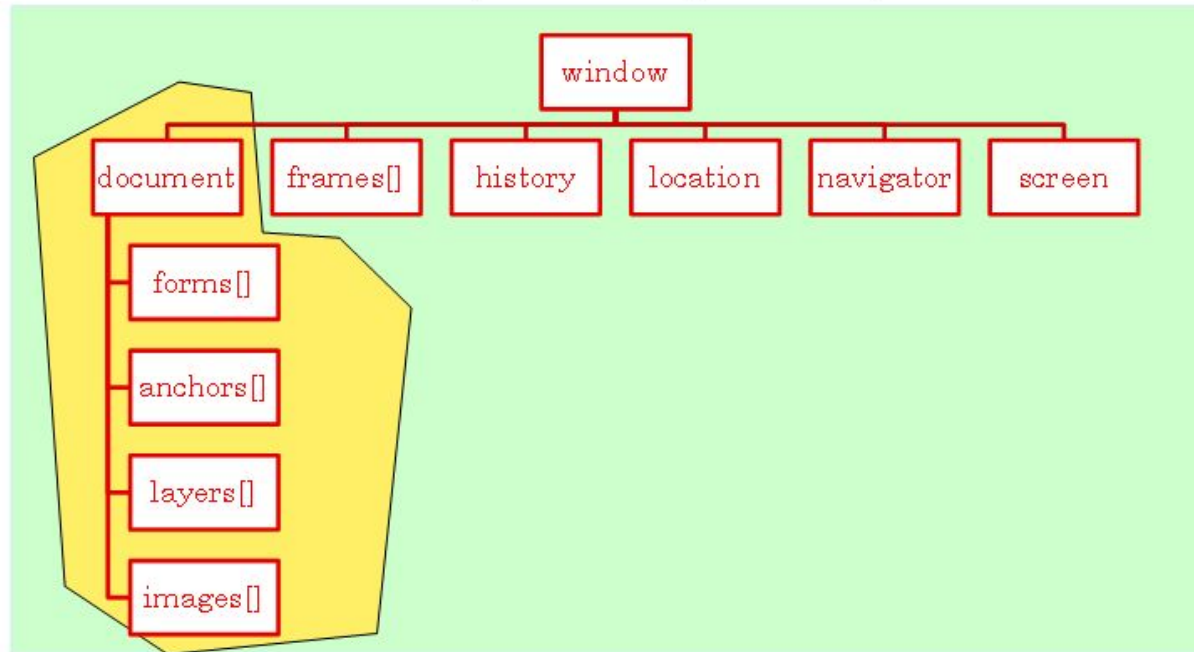
DOM과 HTML 요소 제어하기



JS에서 사용할수있는 내장 객체종류

- HTML 문서를 객체로 표현한 것을 DOM
- 웹 브라우저를 객체로 표현한 것을 BOM(Browser Object Model)

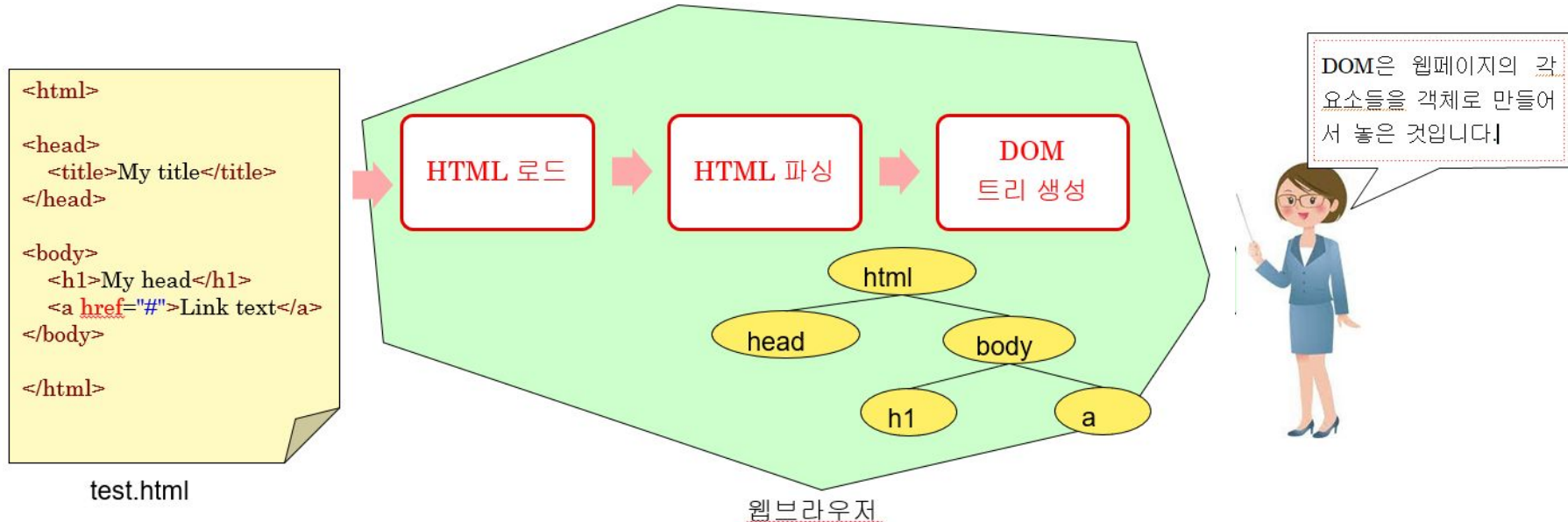
BOM(Browser Object Model)



DOM(Document Object Model)

문서 객체 모델(DOM)

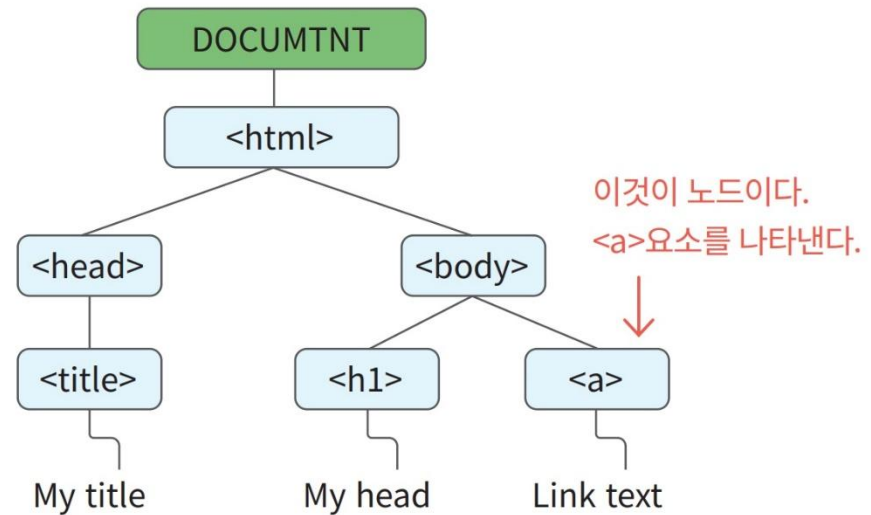
- DOM(Document Object Model)은 HTML 문서의 계층적인 구조를 트리(tree)로 표현
- 자바스크립트가 간편하게 HTML 문서에 접근하여 수정할 수 있도록, HTML 요소를 객체로 만들어 놓는 것이다.



HTML->DOM

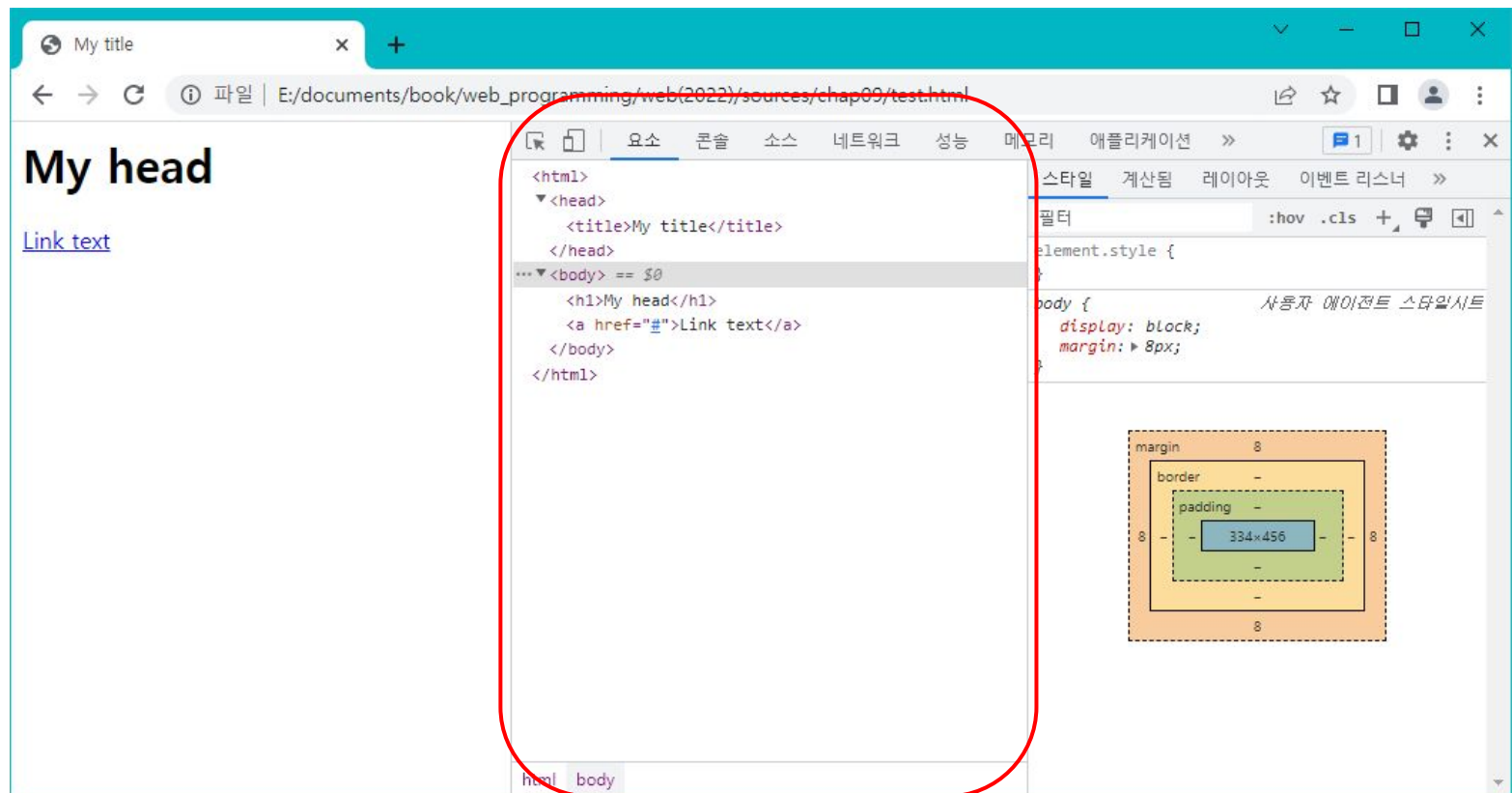
- 웹 페이지가 로드되면, 브라우저는 **DOM**을 생성한 후에 이 **DOM**을 순회하면서 웹페이지를 화면에 표시한다.

```
<html>
  <head>
    <title>My title</title>
  </head>
  <body>
    <h1>My head</h1>
    <a href="#" >Link text</a>
  </body>
</html>
```



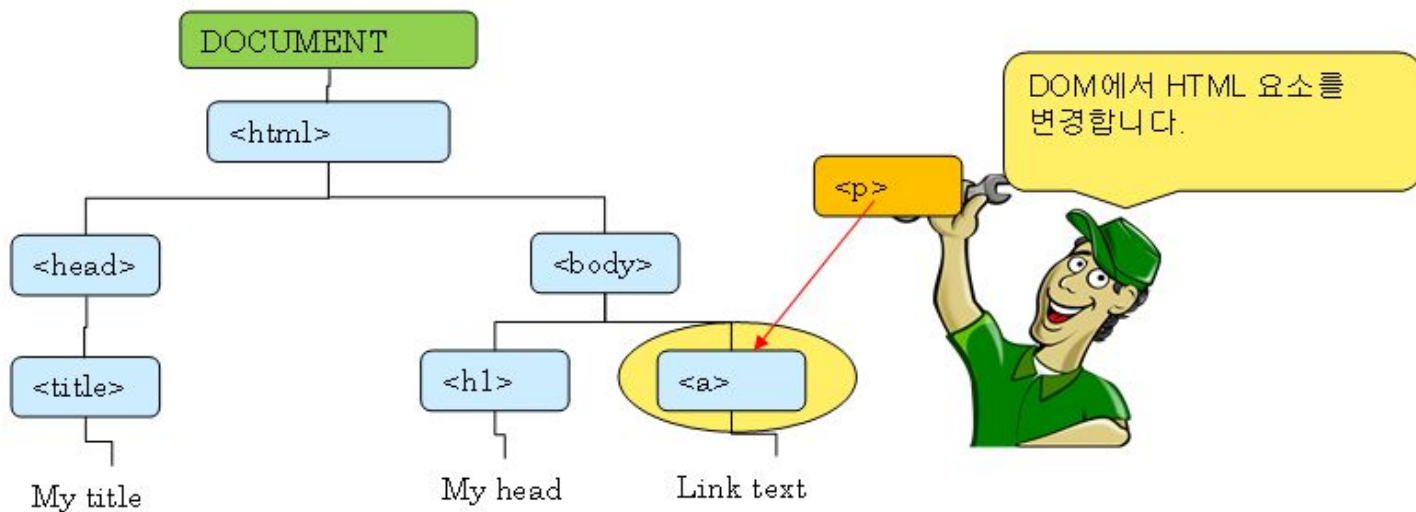
웹 페이지의 DOM 트리 확인하는 방법

- 웹 페이지의 **DOM** 트리는 어디서 볼 수 있을까? 크롬 브라우저에서 마우스 오른쪽 버튼을 누르고 [검사]를 선택하면 다음과 같은 화면이 등장한다



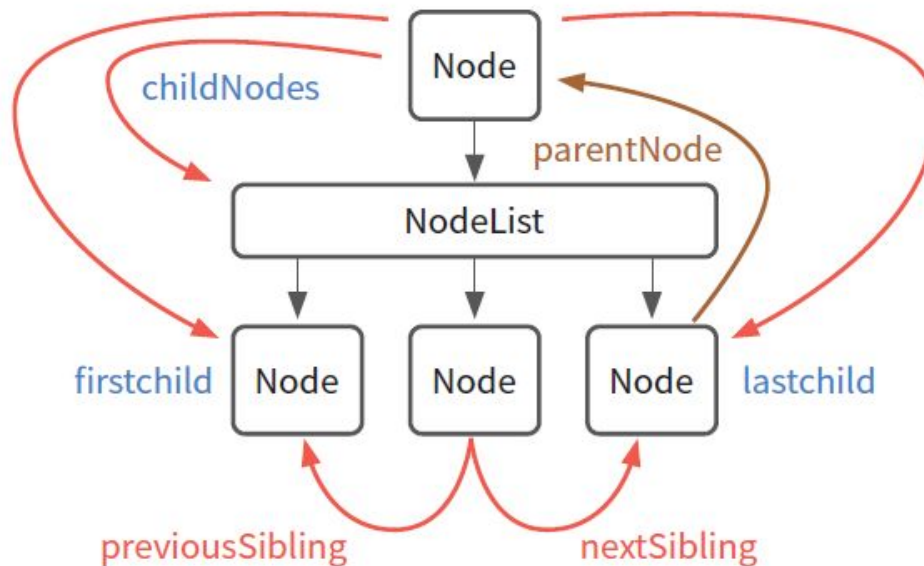
DOM을 이용하여 무엇을 할 수 있을까?

- 우리는 자바스크립트를 사용하여 **DOM** 트리를 순회하면서 페이지 안의 **HTML** 요소들의 내용(콘텐츠)이나 속성, **CSS** 스타일을 우리 마음대로 변경할 수 있다.
- 각각의 요소에서 발생하는 이벤트에 반응하는 코드를 작성할 수 있다.



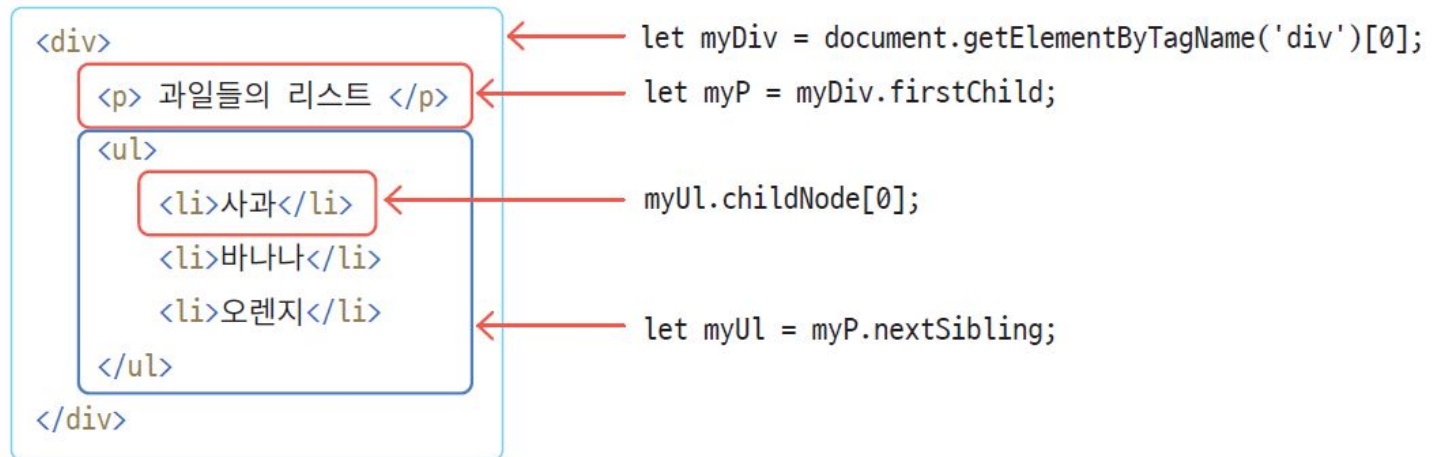
객체들 간의 관계를 이용한 DOM 트리 순회

- DOM 트리를 순회하는 한 가지 방법은 자식 노드와 부모 노드 관계를 이용하여 한 노드씩 방문하는 방법이다. 이 방법에서는 `childNodes[]`, `nextSibling[]`, `parentNode[]`을 이용한다.



childNodes[], nextSibling[], parentNode[]

- **childNodes**: 한 요소의 모든 자식 요소에 접근할 수 있다. 배열이 반환된다.
- **firstChild**: 'childNodes' 배열의 첫 번째 자식 노드가 반환된다. 'childNodes[0]'과 같다.
- **lastChild**: 'childNodes' 배열의 마지막 자식 노드가 반환된다. 'childNodes[childNodes.length-1]'과 같다.
- **parentNode**: 현재 노드의 부모 노드를 반환한다.
- **nextSibling**: 현재 노드의 다음 형제 노드를 반환한다.
- **previousSibling**: 현재 노드의 이전 형제 노드를 반환한다.



DOM 노드 삭제와 추가

- `document` 객체는 DOM 트리의 루트 노드이다. `document` 객체는 HTML 요소를 찾거나 추가하거나 삭제하는 메소드도 제공한다.

메소드	설명
<code>document.createElement(element)</code>	HTML 요소를 생성한다.
<code>document.removeChild(element)</code>	HTML 요소를 삭제한다.
<code>document.appendChild(element)</code>	HTML 요소를 추가한다.
<code>document.replaceChild(new, old)</code>	HTML 요소를 대체한다.
<code>document.write(text)</code>	HTML 출력 스트림에 쓴다.

새로운 HTML 요소 생성

1. 첫 번째 단계는 새로운 요소를 생성하는 것이다. 요소를 생성하는 것은 어렵지 않다. `document` 객체의 `createElement()` 메소드를 호출하면 된다.

```
var node = document.createElement("p");
```

2. 두 번째 단계는 문서(DOM 트리) 내에서 추가할 위치를 찾는 것이다.

```
var parent = document.getElementById("target") ;
```

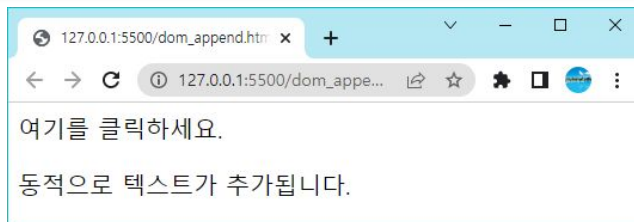
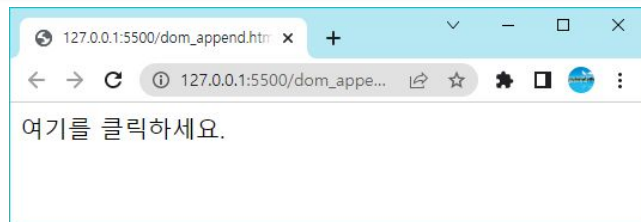
3. 세 번째 단계에서는 새로운 요소를 기존의 요소에 연결한다.

```
parent.appendChild(node);
```

예제

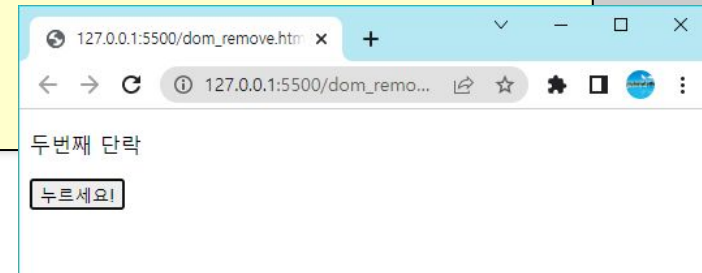
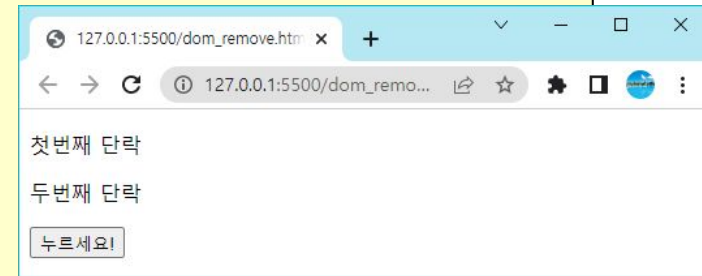
```
<!DOCTYPE html>
<html lang="ko">
<head>
  <script>
    function addNode(t) {
      let node = document.createElement("p");
      node.innerHTML=t;
      document.getElementById("target").appendChild(node);
    }
  </script>
</head>
<body>

<div id="target" onclick="addNode('동적으로 텍스트가 추가됩니다.')"
  style="font: 20px italic;">여기를 클릭하세요.</div>
</body>
</html>
```



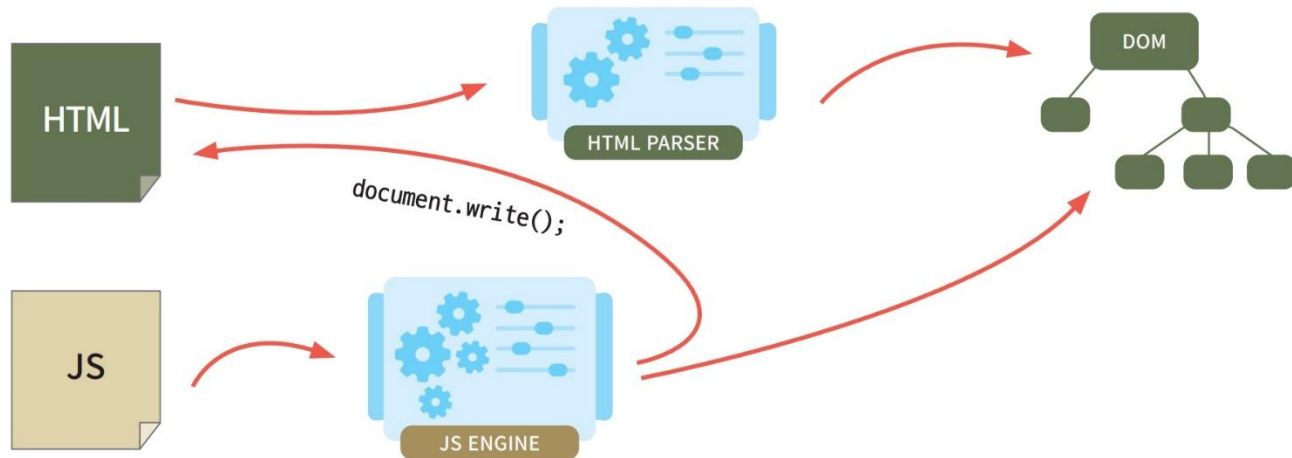
기존의 HTML 요소를 삭제

```
<!DOCTYPE html>
<html>
  <head>
    <script>
      function removeNode() {
        let parent = document.getElementById("target");
        let child = document.getElementById("p1");
        parent.removeChild(child);
      }
    </script>
  </head>
  <body>
    <div id="target">
      <p id="p1">첫번째 단락</p>
      <p id="p2">두번째 단락</p>
    </div>
    <button onclick="removeNode()">누르세요!</button>
  </body>
</html>
```



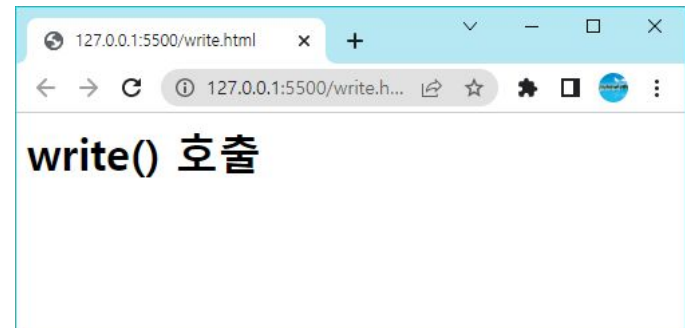
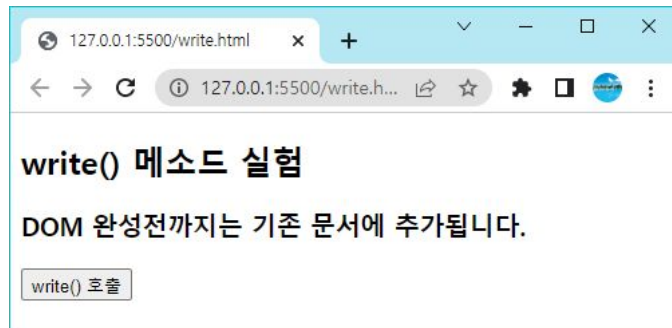
document.write() 메소드

- DOM 트리 생성 종료 이전에 **write()** 메소드를 만나면 큰 문제 없이, 현재 작업 중인 DOM 트리에 주어진 콘텐츠를 추가한다. 하지만 DOM 트리가 완전히 생성된 후에 **write()**를 호출하면 기존의 DOM가 모두 삭제된다. 따라서 주의하여야 한다.



예제

```
<!DOCTYPE html>
<html>
<body>
  <h2>write() 메소드 실험 </h2>
  <script>
    document.write("<h3> DOM 완성전까지는 기존 문서에 추가됩니다.</h3>");
  </script>
  <button onclick="sub()">write() 호출 </button>
  <script>
    function sub() {
      document.write("<h1>write() 호출</h1>");
    }
  </script>
</body>
</html>
```



DOM제어를 위한 HTML 요소 찾기

- DOM노드 객체들의 관계가 아니어도 HTML 요소를 제어할 수 있다.
- 동적인 웹페이지를 작성하려면 원하는 요소를 찾아야 한다.
 - id로 HTML 요소 찾기
 - 태그 이름으로 HTML 요소 찾기
 - 클래스 이름으로 HTML 요소 찾기
 - CSS 선택자를 이용하여 HTML 요소 찾기

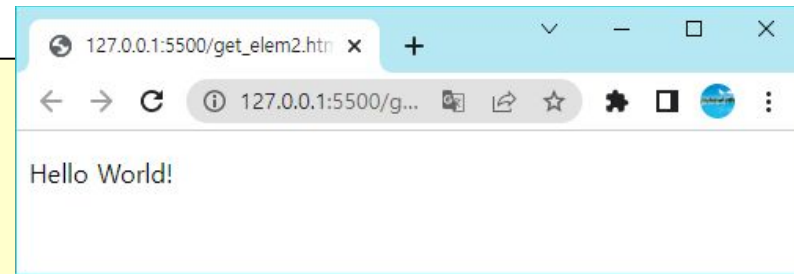
id로 HTML 요소 찾기

- DOM에서 HTML 요소를 찾는 가장 쉬운 방법은 요소의 id를 이용하는 것이다.

```
x = document.getElementById("main");
```

→ 문서 안에서 id가 "main"인
요소를 찾아서 반환한다.

```
<!DOCTYPE html>
<html>
<body>
  <p id="special"></p>
  <script>
    document.getElementById("special").innerHTML
      = "Hello World!";
  </script>
</body>
</html>
```



태그 이름으로 HTML 요소 찾기

- 비슷한 메소드로 `getElementsByTagName()` 메소드가 있다. 이 메소드는 태그의 이름을 인수로 받아서 이 태그를 사용하는 모든 요소들을 배열에 넣어서 반환한다.

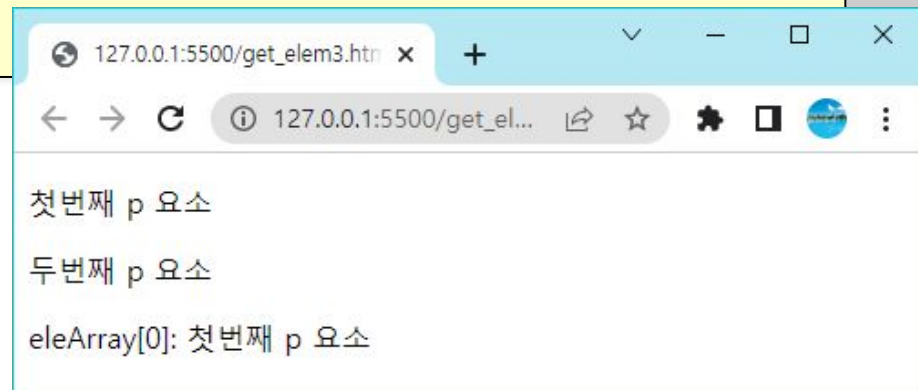
```
eleArray = document.getElementsByTagName("p");
```

→ 문서 안의 <p> 태그를
모두 찾아서 반환한다.

→ 배열로 반환된다. 첫 번째 요소는 `eleArray[0]`이다.

태그 이름으로 HTML 요소 찾기

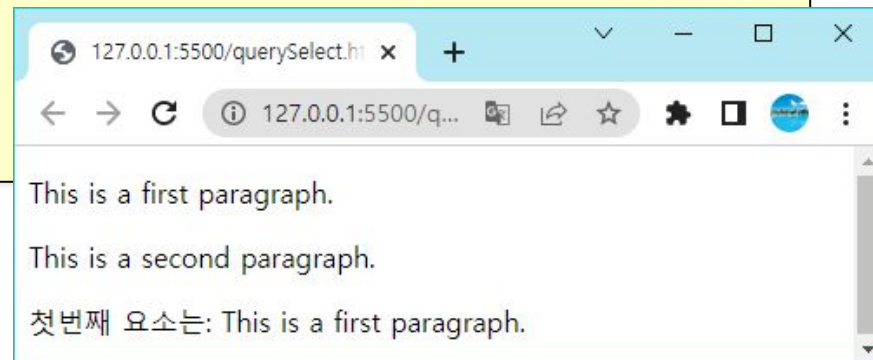
```
<!DOCTYPE html>
<html>
<body>
  <p>첫번째 p 요소</p>
  <p>두번째 p 요소</p>
  <p id="special"></p>
  <script>
    var eleArray = document.getElementsByTagName("p");
    document.getElementById("special").innerHTML = 'eleArray[0]: ' + eleArray[0].innerHTML;
  </script>
</body>
</html>
```



CSS 선택자로 HTML 요소 찾기

- 지정된 CSS 선택자와 일치하는 모든 HTML 요소를 찾으려면 `querySelectorAll()` 메소드를 사용한다.

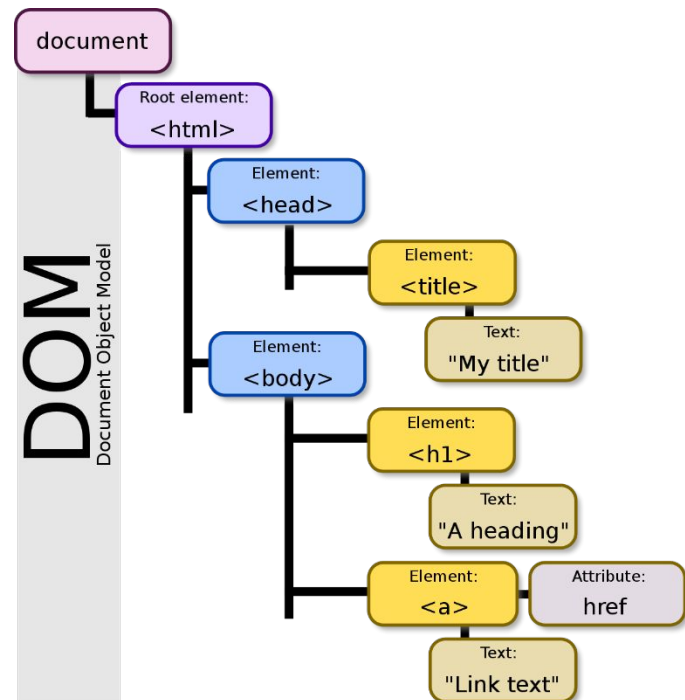
```
<!DOCTYPE html>
<html>
<body>
  <p class="special">This is a first paragraph.</p>
  <p class="special">This is a second paragraph.</p>
  <p id="result"></p>
  <script>
    var arr = document.querySelectorAll("p.special");
    document.getElementById("result").innerHTML =
      '첫번째 요소는: ' + arr[0].innerHTML;
  </script>
</body>
</html>
```



HTML 요소 변경하기

- 자바스크립트를 이용하여 HTML 문서의 DOM을 변경하는 것은 3가지 경우로 나눌 수 있다. HTML 문서가 변경되는 것은 아니고 DOM만 변경된다는 것을 잊지 말자.

- 요소의 내용 변경하기
- 요소의 속성 변경하기
- 요소의 스타일 변경하기



요소의 내용 변경하기

- HTML 요소의 내용을 변경하는 가장 쉬운 방법은 `innerHTML` 속성을 사용하는 것이다. `innerHTML`은 요소의 시작 태그와 종료 태그 사이에 놓여진 모든 HTML 코드와 텍스트를 의미한다.

여기가 바로 `<div>` 요소의 `innerHTML`이다.

`<div id="main">`이것이 div 요소입니다.`</div>`

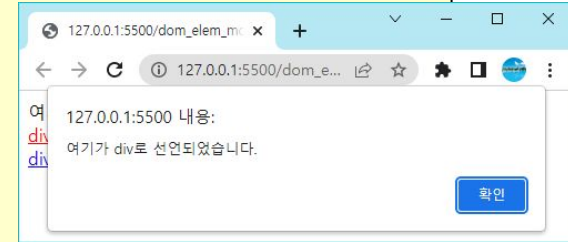
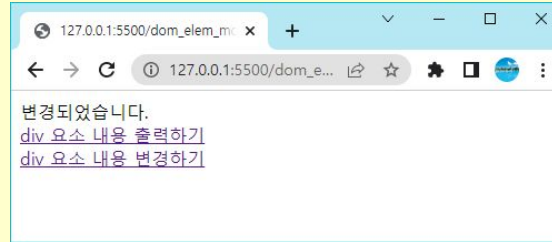
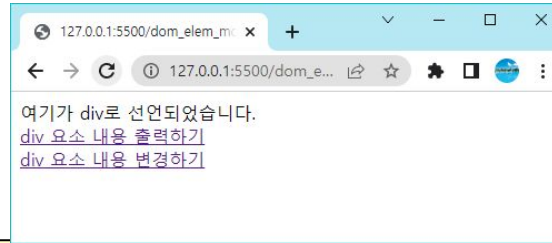
시작 태그 ← → 종료 태그

```
document.getElementById("main").innerHTML = "웹페이지가 작성되었습니다.";
```

```
<div id="main">  
    이것이 div 요소입니다.  
</div>
```

```
<div id="main">  
    웹페이지가 작성되었습니다.  
</div>
```

예제



```
<!DOCTYPE html>
<html>
<head>
  <title></title>
  <script>
    function get() {
      let val = document.getElementById("ex").innerHTML;
      alert(val);
    }
    function set(v) {
      document.getElementById("ex").innerHTML = v;
    }
  </script>
</head>

<body>
  <div id="ex">여기가 div로 선언되었습니다.</div>
  <a href="#" onclick="get()">div 요소 내용 출력하기</a><br />
  <a href="#" onclick="set('변경되었습니다.')">div 요소 내용 변경하기</a>
</body>
</html>
```

요소의 속성 변경하기

- HTML 요소의 속성을 변경하려면 다음과 같은 문법을 사용한다.

속성 이름 ← → 새로운 속성값

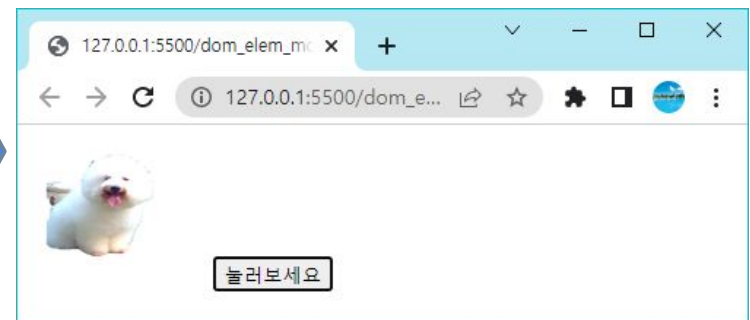
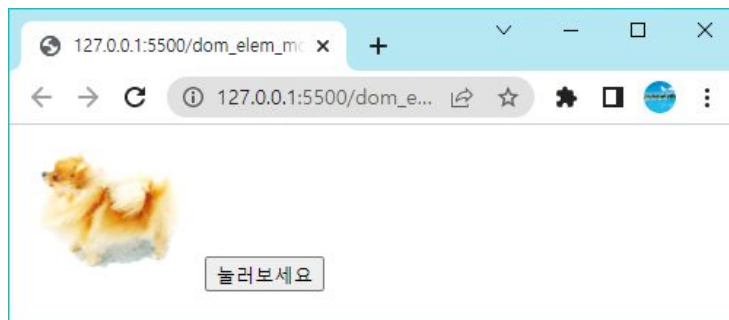
```
document.getElementById("image").src = "poodle.png";
```

예제

```
<!DOCTYPE html>
<html>
<body>

  
  <script>
    function changeImage() {

      document.getElementById("image").src = "poodle.png";
    }
  </script>
  <input type="button" onclick="changeImage()" value="눌러보세요" />
</body>
</html>
```



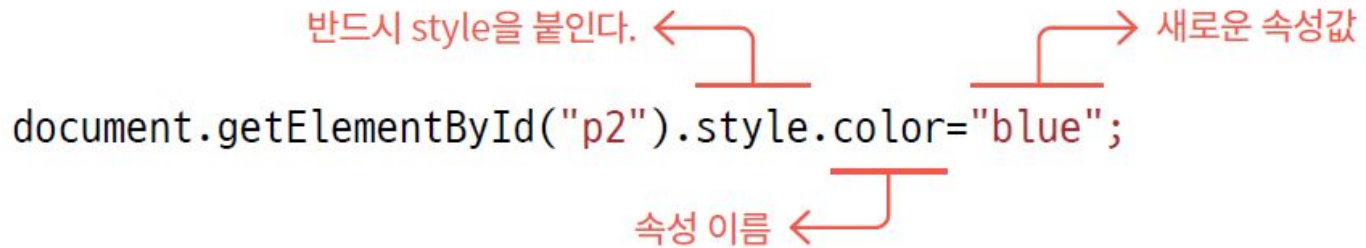
요소의 스타일 변경하기

- DOM과 자바스크립트를 사용하면 HTML 요소의 스타일도 얼마든지 동적으로 변경할 수 있다.

반드시 style을 붙인다. ← 새로운 속성값

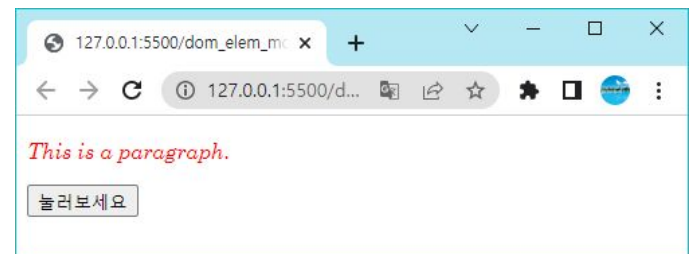
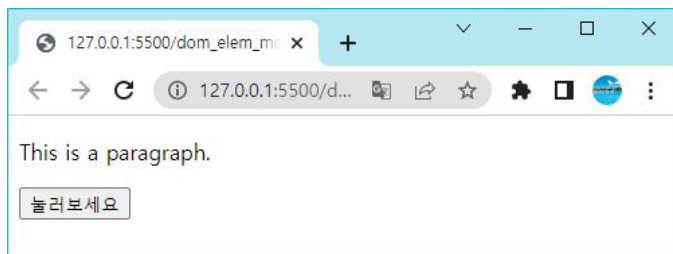
```
document.getElementById("p2").style.color="blue";
```

← 속성 이름



예제 1.

```
<!DOCTYPE html>
<html>
<body>
<p id="p1">This is a paragraph.</p>
<script>
  function changeStyle() {
    document.getElementById("p1").style.color = "red";
    document.getElementById("p1").style.fontFamily = "Century Schoolbook";
    document.getElementById("p1").style.fontStyle = "italic";
  }
</script>
  <input type="button" onclick="changeStyle()" value="눌러보세요" />
</body>
</html>
```



예제 2.

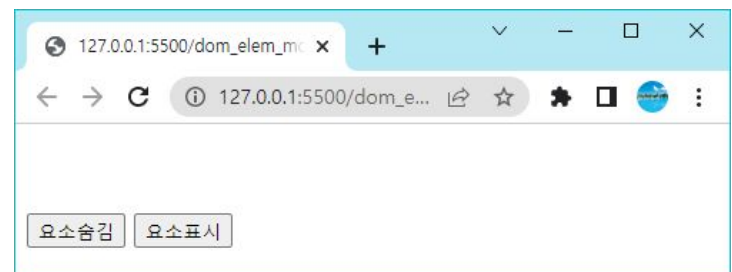
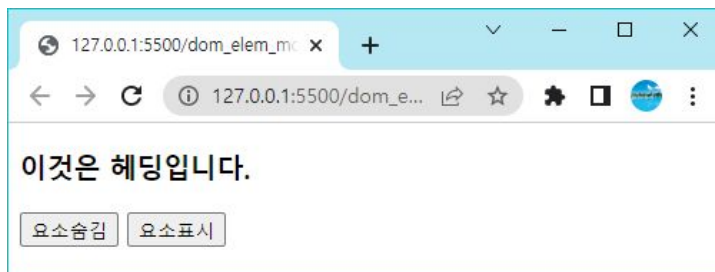
```
<!DOCTYPE html>
<html>
<body>

<h3 id="heading">이것은 heading입니다.</h3>

<input type="button" value="요소숨김"
onclick="document.getElementById('heading').style.visibility = 'hidden'" />

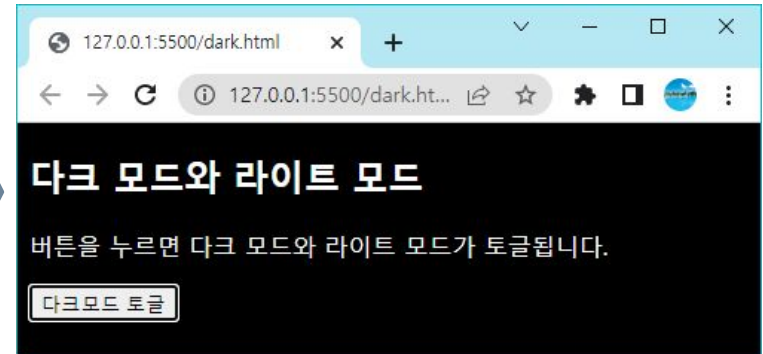
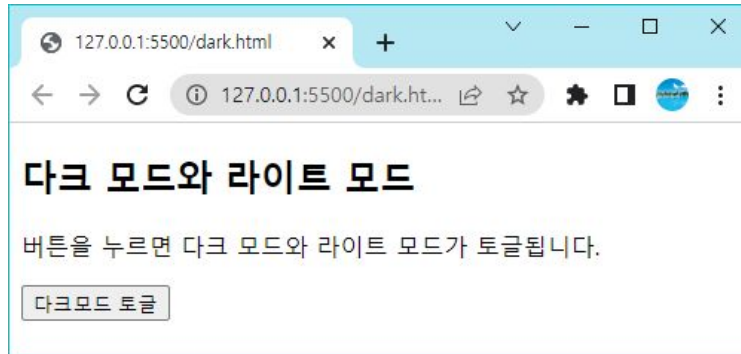
<input type="button" value="요소표시"
onclick="document.getElementById('heading').style.visibility = 'visible'" />

</body>
</html>
```



예제 3.

- 자바스크립트를 이용하여서 다크 모드와 라이트 모드를 토글 할 수 있는 버튼을 만들어보자.



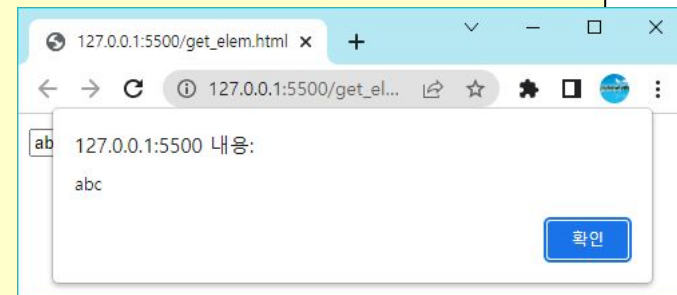
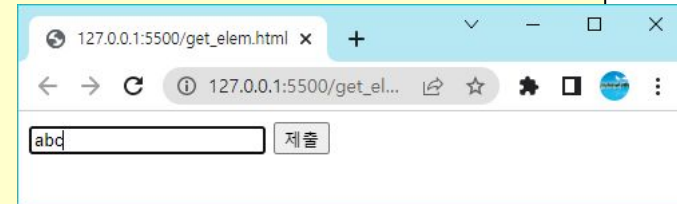
예제 3.

```
<!DOCTYPE html>
<html>
<head>
  <script>
    let mode = "white";
    function sub() {
      if (mode == "white") {
        document.getElementsByTagName("body")[0].style.backgroundColor = "black";
        document.getElementsByTagName("body")[0].style.color = "white";
        mode = "black";
      } else {
        document.getElementsByTagName("body")[0].style.backgroundColor = "white";
        document.getElementsByTagName("body")[0].style.color = "black";
        mode = "white";
      }
    }
  </script>
</head>
<body>
  <h2>다크 모드와 라이트 모드</h2>
  <p>버튼을 누르면 다크 모드와 라이트 모드가 토글됩니다.</p>
  <button onclick="sub()">다크모드 토글</button>
</body>
</html>
```

입력 양식에서 사용자 입력값 읽기 #1

- 일반 HTML 요소에서는 innerHTML 속성을 이용하여 요소 안의 내용을 읽으면 된다. 하지만 <input>과 같은 입력 양식에서는 어떻게 읽어야 할까?

```
<!DOCTYPE >
<html>
<head>
  <script>
    function process() {
      let obj = document.getElementById("target");
      alert(obj.value);
    }
  </script>
</head>
<body>
  <form name="myform">
    <input type="text" id="target" name="text1">
    <input type="submit" value="제출" onclick="process()">
  </form>
</body>
</html>
```



입력 양식에서 사용자 입력값 읽기 #2

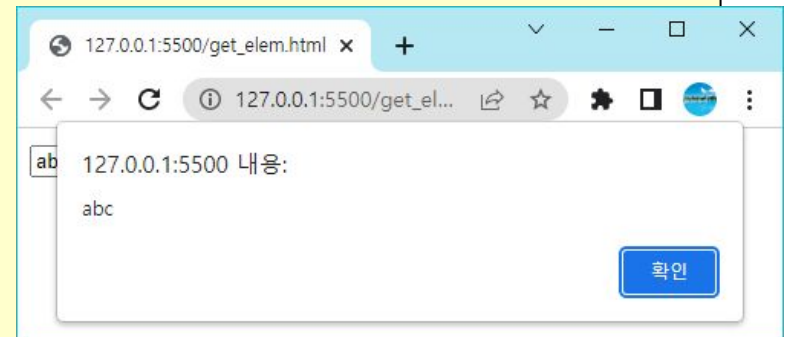
- 입력 요소를 찾을 때, 많이 사용되는 방법이 하나 더 있다. **document** 객체 안에 **forms**라는 배열 객체가 있다. **forms** 객체는 **document** 객체 안에 존재하는 모든 입력 양식을 저장한다.

```
<form> ← document.forms[0]
  <input type="text" value=''> ← document.forms[0].elements[0]
  <input type="text" value=''> ← document.forms[0].elements[1]
  <input type="submit">
</form>
```

```
<form name="myform"> ← document.myform
  <input type="text" id="target1" name="text1"> ← document.myform.text1
  <input type="text" id="target2" name="text2"> ← document.myform.text2
  <input type="submit">
</form>
```

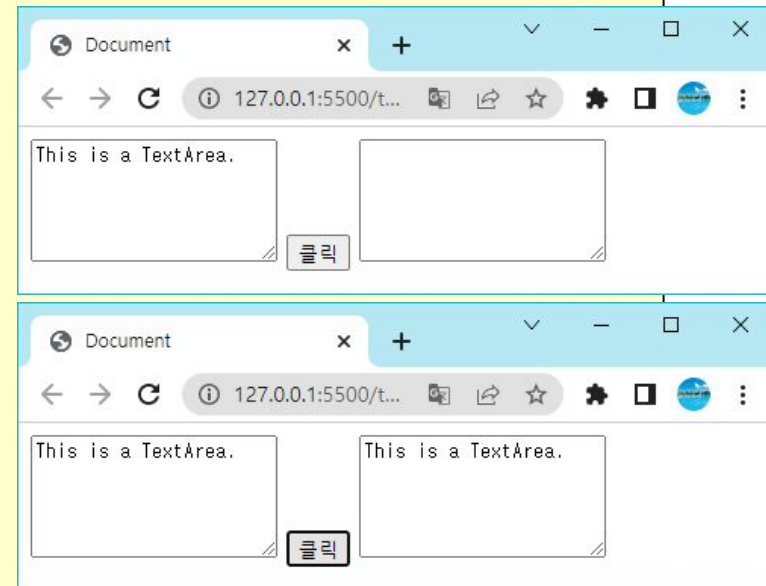
예제

```
<!DOCTYPE >
<html>
<head>
  <script>
    function process() {
      var obj = document.myform.text1;
      alert(obj.value);
    }
  </script>
</head>
<body>
  <form name="myform">
    <input type="text" id="target" name="text1">
    <input type="submit" value="제출" onclick="process()">
  </form>
</body>
</html>
```



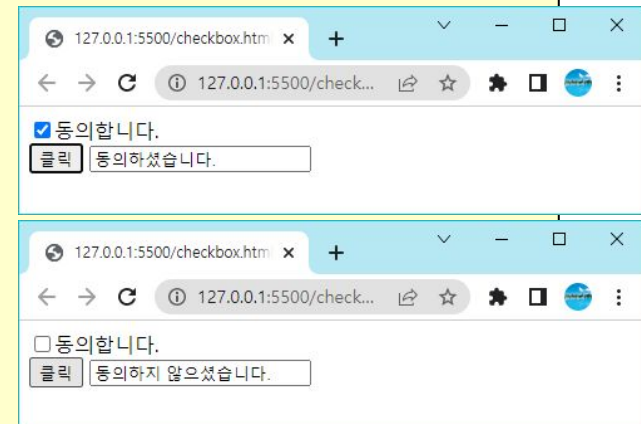
텍스트 영역에서 읽기

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <script>
    function getTextAreaValue() {
      var s;
      s = document.getElementById("textArea").value;
      document.getElementById("result").value = s;
    }
  </script>
</head>
<body>
  <TEXTAREA id="textArea" rows=5 cols=20>This is a TextArea.</TEXTAREA>
  <input type="button" onClick="getTextAreaValue();" value="클릭">
  <TEXTAREA id="result" rows=5 cols=20></TEXTAREA>
</body>
</html>
```



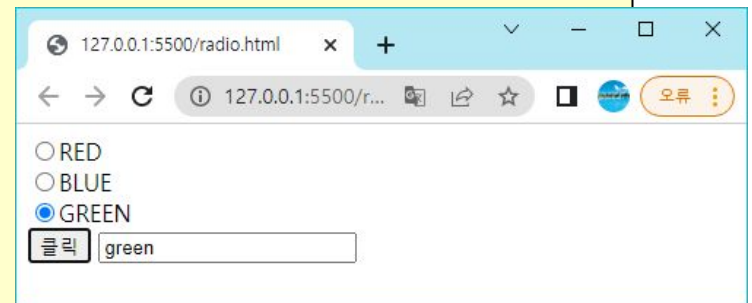
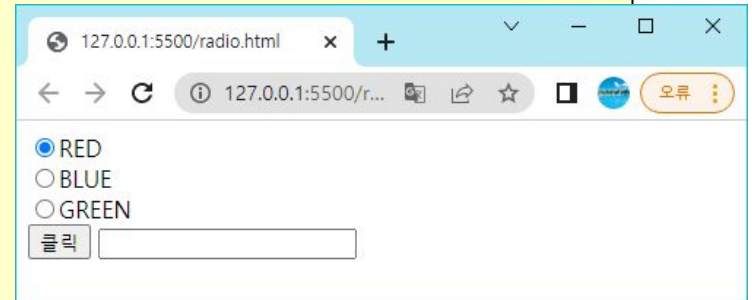
체크 박스에서 읽기

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <script>
    function get() {
      if (document.getElementById("myCheckBox").checked) {
        document.getElementById("result").value = "동의하셨습니다.";
      } else {
        document.getElementById("result").value = "동의하지 않으셨습니다.";
      }
    }
  </script>
</head>
<body>
  <input type="checkbox" id="myCheckBox">동의합니다.<br>
  <input type="button" onClick="get();" value="클릭">
  <input type="text" id="result" value="">
</body>
</html>
```



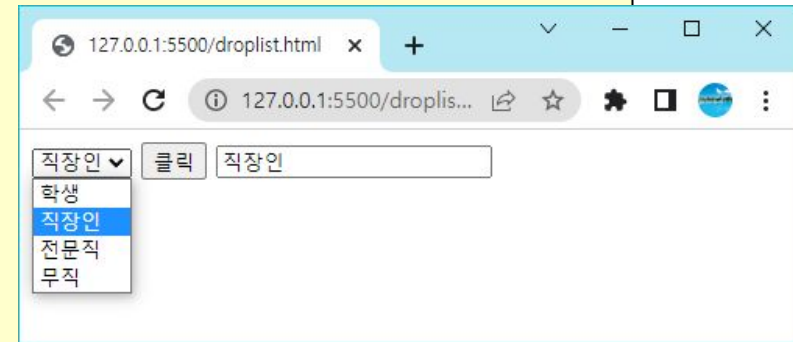
라디오 버튼에서 읽기

```
<!DOCTYPE html>
<html lang="en">
<head>
  <script>
    function getValue() {
      let list = document.getElementsByName('color');
      for (let i = 0; i < list.length; i++) {
        if (list[i].checked) {
          document.getElementById("result").value = list[i].value;
          break;
        }
      }
    }
  </script>
</head>
<body>
  <input type="radio" name="color" CHECKED value="red">RED<BR>
  <input type="radio" name="color" value="blue">BLUE<BR>
  <input type="radio" name="color" value="green">GREEN<BR>
  <input type="button" onClick="getValue();" value="클릭">
  <input type="text" id="result" value="">
</body>
</html>
```



SelectList에서 값읽기

```
<!DOCTYPE html>
<html lang="en">
<head>
  <script>
    function getValue() {
      var selected, s;
      selected = document.getElementById("myList").selectedIndex;
      s = document.getElementById("myList").options[selected].value;
      document.getElementById("result").value = s;
    }
  </script>
</head>
<body>
  <select id="myList">
    <option value="학생">학생</option>
    <option value="직장인" SELECTED>직장인</option>
    <option value="전문직">전문직</option>
    <option value="무직">무직</option>
  </select>
  <input type="button" onClick="getValue()" value="클릭">
  <input type="text" id="result" value="">
</body>
</html>
```



Q & A

